Detailed Documentation: End-to-End Data Management Pipeline for Machine Learning

By

Group: 31

Amulya Gupta	2024AB05200
Navya Saxena	2024AA05234
Ravi Bhardwaj	2024AA05356

Prepared in Partial Fulfillment of the Course

Data Management for Machine Learning

(AIMLCZG529/DSECLZG529)(S2-24)

Birla Institute of Technology and Science (BITS), Pilani

August, 2025

Table of Contents

Pipeline Architecture:

The pipeline is designed as a structured, step-by-step flow that takes data from raw ingestion all the way to model training and evaluation. Each stage builds on the previous one, ensuring data quality, reproducibility, and automation. Below, we outline each component of the architecture to show how they connect together into a complete end-to-end workflow.

1. Problem Formulation

1.1 Business Problem Definition:

The business problem is to predict customer churn—that is, to identify customers who are likely to leave or discontinue the company's services in the near future. Churn prediction is crucial in industries such as telecommunications, banking, and subscription-based services, where retaining customers has a direct impact on profitability and long-term growth.

1.2 Key Business Objectives

- Increase Customer Retention: Identify the risk of losing customers at the initial stage and implement targeted retention strategies.
- Reduce Revenue Loss: Minimize churn-driven revenue decline through timely interventions.
- Optimize Marketing Spend: Focus marketing and incentives on segments most likely to churn.
- Improve Service Quality: Understand churn drivers to address product or service gaps.

In this stage, we integrated all the previous steps (Points 1 to 9) into a single end-to-end pipeline. The primary goal of orchestration is to automate the workflow, ensuring that every step is executed in the correct order without manual intervention. The pipeline begins with data ingestion and validation, proceeds through preparation and feature transformation, publishes features into the feature store, handles versioning, and concludes with model training and evaluation.

We used Prefect as the orchestration tool because it is lightweight, easy to configure in Colab, and provides features like task retries, logging, and scheduling. Each notebook/script from earlier steps was converted into a Prefect task and then combined into a flow. The flow also includes important safety checks, such as failing early if validation errors occur or if the model’s ROC AUC score is below a set threshold.

The orchestration produces a run summary JSON file, which records all the important artifacts (raw data, validated datasets, transformed features, feature snapshots, trained models, and evaluation reports). This ensures the workflow is not only automated but also reproducible and easy to trace. With orchestration in place, the entire pipeline can be executed reliably with a single command, and it can also be scheduled to run automatically on a daily or weekly basis in the future.

1.3 Key Data Sources and Their Attributes

1.3.1 Telco Customer Churn Dataset (Kaggle, IBM Sample Data):

- Description: Records demographics, account information, service subscriptions, and payment details of 7,043 telecom customers. The target variable is Churn (Yes/No), indicating whether the customer left the service.
- Attributes:
  - ➢ customerID – Unique identifier for each customer
  - ➢ gender – Male/Female
  - ➢ SeniorCitizen – 1 = Yes,
  - ➢ customerID – Unique identifier for each customer
  - ➢ gender – Male/Female
  - ➢ SeniorCitizen – 1 = Yes, 0 = No
  - ➢ Partner – Yes/No (customer has a partner)
  - ➢ Dependents – Yes/No (customer has dependents)
  - ➢ tenure – Number of months the customer has stayed with the company
  - ➢ PhoneService – Yes/No
  - ➢ MultipleLines – Yes/No/No phone service
  - ➢ InternetService – DSL/Fiber optic/No
  - ➢ OnlineSecurity – Yes/No/No internet service
  - ➢ OnlineBackup – Yes/No/No internet service
  - ➢ DeviceProtection – Yes/No/No internet service
  - ➢ TechSupport – Yes/No/No internet service
  - ➢ StreamingTV – Yes/No/No internet service
  - ➢ StreamingMovies – Yes/No/No internet service
  - ➢ Contract – Month-to-month/One-year/Two-year
  - ➢ PaperlessBilling – Yes/No
  - ➢ PaymentMethod – Electronic check/Mailed check/Bank transfer/Credit card (automatic)
  - ➢ TotalCharges – Total amount billed over tenure

1.3.2 Generic Customer Churn Dataset:

- Description: Contains structured tabular data where each row represents an individual customer and their service attributes, demographic information, and churn status.
- Attributes:
  - ➢ customerID-Unique identifier for each customer (e.g., Cust_1, Cust_2, ...).
  - ➢ gender- Sex of customer (Male, Female).
  - ➢ SeniorCitizen- Indicates if the customer is a senior citizen (0: No, 1: Yes)
  - ➢ Partner- Indicates if the customer has a partner (Yes, No)
  - ➢ Dependents- Indicates if the customer has dependents (Yes, No)
  - ➢ Tenure- Number of months the customer has stayed with the company (integer values)
  - ➢ PhoneService- Whether the customer subscribes to phone service (Yes, No)
  - ➢ MultipleLines- If the customer has multiple phone lines (Yes, No, No phone service)

- ➢ InternetService- Type of internet service (DSL, Fiber optic, No, No internet service)
- ➢ OnlineSecurity- Whether the customer subscribes to online security add-on (Yes, No, or No internet service)
- ➢ OnlineBackup- Whether the customer subscribes to online backup add-on (Yes, No, or No internet service)
- ➢ Churn- Target variable indicating if the customer has churned (Yes, possibly missing for some rows)

1.4 Expected Outputs from the Pipeline

1.4.1 Clean Datasets for Exploratory Data Analysis (EDA)

- Description: Preprocessed and validated datasets with consistent formatting, imputed or handled missing values, filtered duplicates, and corrected data types.
- Source: Data validation and cleansing scripts produce these clean datasets (e.g., Task 4 data validation and preparation).
- Format: CSV files with cleaned columns, stored under structured raw or clean data folders.
- Purpose: Ready for EDA activities such as summary statistics, visualizations, and hypothesis generation.

1.4.2 Transformed Features for Machine Learning

- Description: Engineered features including derived columns (e.g., tenure buckets, spend per month, support calls per month), scaled and encoded numeric and categorical attributes, and aggregated customer behavior indicators.
- Source: Feature engineering and transformation scripts (Task 5 and Task 6) generate these features.
- Format: Timestamped CSV snapshots persisted in a feature store directory with accompanying metadata files.
- Purpose: Direct input to ML model training, ensuring feature consistency and reproducibility.

1.4.3 A Deployable Model to Predict Customer Churn

- Description: Trained machine learning models capable of predicting the likelihood of customer churn based on transformed features.
- Source: Model training scripts (Task 9) train and serialize models such as Logistic Regression and Random Forest classifiers.
- Format: Persisted model files (e.g., .pkl files) saved with versioned timestamps and associated evaluation reports.
- Purpose: Ready for integration into downstream applications for batch inference or deployment as a service to predict churn likelihood.

1.4.4 Measurable Evaluation Metrics

Evaluation metrics captured in the model training and reporting stage include:

- Accuracy: The overall percentage of correctly predicted customers (both churned and not churned). Useful as a general measure, but can be misleading if the class distribution is imbalanced.
- Precision (Positive Predictive Value): The proportion of customers predicted to churn who actually churn. High precision means fewer false alarms, which is important to avoid wasting retention resources on customers unlikely to leave.
- Recall (Sensitivity or True Positive Rate): The proportion of actual churned customers correctly identified by the model. High recall ensures that the majority of churners are caught early, enabling effective intervention.
- F1 Score: The harmonic mean of precision and recall, providing a balanced metric when classes are imbalanced. Useful when both false positives and false negatives have significant business costs.
- ROC-AUC (Area Under the Receiver Operating Characteristic Curve): Measures the model's ability to distinguish between churn and non-churn across all classification thresholds. Closer to 1 indicates better discriminative power; valuable for assessing overall model performance beyond a single threshold.
- Confusion Matrix: A detailed matrix showing true positives, true negatives, false positives, and false negatives. Helps understand errors and guide business strategies for handling different types of misclassifications.
- Cost-sensitive Metrics: Incorporate business costs of false positives (e.g., cost of unnecessary retention efforts) and false negatives (loss of customer revenue). Metrics such as expected monetary value or cost-benefit analysis can be useful to contextualize predictions monetarily.

These metrics are systematically logged and reported for each trained model to support transparent evaluation and comparison.

2. Data Ingestion

2.1 Automated Fetching of Data from Sources (APIs)

- Telco Customer Churn (Kaggle):
    - The notebook installs and configures the Kaggle API, then programmatically downloads and unzips the dataset into /content/kaggledata.
    - Path used post-download: /content/kaggledata/WA_Fn-UseC_-Telco-Customer-Churn.csv.
    - This constitutes automated fetching from an external data source.

- Generic Customer Churn (Hugging Face):

    - The notebook authenticates to Hugging Face and uses datasets.load_dataset("drMurder/customer_churn_dataset", split="train") to fetch the dataset.
    - The dataset is converted to a pandas DataFrame and saved to /content/hfdata/generic_customer_churn.csv for uniform ingestion.
    - This constitutes automated fetching via an API client.

- Ingestion Function:
    - ingest_csv(source_path, target_dir) reads a CSV (from the above sources) and writes a timestamped raw snapshot into a source-partitioned directory under /content/raw.

- Optional scheduling:
    - A separate ingestion job pattern exists (in prior work) to run ingestion periodically (e.g., daily), which can be easily applied to these paths if persistent execution is available.

What this achieves:

- Programmatic downloads from APIs (Kaggle and Hugging Face) rather than manual uploads.
- Consistent conversion (HF → CSV) so the same ingestion function handles both sources.

2.2 Error Handling for Failed Ingestions

- Try/except around each source:
    - Kaggle download and ingestion are wrapped in a try/except block; failures are caught and logged: “Kaggle download or ingestion failed: ...”
    - Hugging Face download and ingestion are wrapped in a try/except block; failures are caught and logged: “Hugging Face download or ingestion failed: ...”
- ingest_csv uses try/except:
    - Any failure to read/write a CSV logs a detailed error and returns False to signal the failure upstream.
- This ensures the pipeline reports failures without crashing the entire run and allows partial success across sources.

2.3 Logging Mechanisms for Tracking Ingestion Status

- Logging configuration:
    - Handlers are explicitly set up to log both to a file and to the console:
        - FileHandler('data_ingestion.log', mode='a')
        - StreamHandler()
    - Existing handlers are cleared at the start to ensure predictable behavior in the notebook environment.
- Logged events include:
    - Start of ingestion from each source path
    - Successful save with fully qualified file path (including timestamp)
    - Any exception during download or ingestion with the error message

- Output log file:
    - The data_ingestion.log file is generated in the working directory and can be attached as a deliverable.

Deliverables produced for Task 2:

- Python ingestion code that fetches from APIs (Kaggle, Hugging Face), ingests with pandas, and persists raw snapshots.
- A log file (data_ingestion.log) with INFO and ERROR entries demonstrating execution status and outcomes.
- Resulting timestamped raw files stored under /content/raw/teldataco and /content/raw/generic (screenshots can be taken from Colab file explorer or via listing commands).

3. Raw Data Storage

3.1 Data Stored in a Data Lake (Google Drive)

- The notebook mounts Google Drive and writes raw files to a durable "data lake" location:
    - Telco raw target: /content/drive/MyDrive/DMML_Project/data/raw/telco
    - Generic raw target: /content/drive/MyDrive/DMML_Project/data/raw/generic
- The helper ingest_and_upload(source_path, target_dir) reads the CSV, ensures the directory exists, and writes a timestamped raw CSV to Drive.
- This provides persistence beyond the Colab session and serves as the project's raw data layer, aligned with a data lake pattern.

3.2 Partitioning by Source, Type, and Timestamp

- Partition by source:
    - Separate subdirectories for each dataset:
        - Telco/ for the Telco Customer Churn data
        - generic/ for the Generic Customer Churn data

- Partition by type:
    - All raw snapshots reside under data/raw/, clearly separating the raw layer from future processed/cleaned layers (e.g., data/processed).

- Partition by timestamp:
    - Each ingested file is named raw_data_YYYYMMDD_HHMMSS.csv, capturing the exact ingestion time and providing versioning for reproducibility and auditability.

Documented structure:

- In Drive:

    - /content/drive/MyDrive/DMML_Project/data/raw/telco/raw_data_YYYYMMDD_HHMMSS.csv
    - /content/drive/MyDrive/DMML_Project/data/raw/generic/raw_data_YYYYMMDD_HHMMSS.csv

- In runtime:
    - /content/raw/teldataco/raw_data_YYYYMMDD_HHMMSS.csv
    - /content/raw/generic/raw_data_YYYYMMDD_HHMMSS.csv

Benefits:

- Efficient retrieval by source and time window
- Clear lineage between ingestion runs
- Scalable and ready for additional sources or layers

4. Data Validation

4.1 Validation checks implemented (missing values, data types, anomalies)

In the script, the validation pipeline for the Generic dataset performs:

- Safe Copying
    - Operates on a working copy so the raw master remains unchanged (df_generic → working copy).
- Missing Values
    - generic_missing = generic.isna().sum().to_dict() records missing counts per column.
    - The report structure includes per-column missing counts under "missing".
- Duplicates
    - If CustomerID exists: generic.duplicated(subset=["CustomerID"]).sum() is used; otherwise generic.duplicated().sum().
    - Duplicates reported as a single number under "duplicates".
- Data Type and Numeric Casting
    - to_numeric_inplace(df, cols) attempts numeric conversion for specified numeric columns (Age, Tenure, Usage Frequency, Support Calls, Payment Delay, Total Spend, Last Interaction).
    - Coercion statistics (values coerced to NaN) are tracked under "numeric_cast".
- Range Validation
    - generic_ranges defines acceptable ranges, e.g.:
        - Age: 0–120
        - Tenure: 0–1000
    - Violations computed and captured per column as {"violations": count, "lo": ..., "hi": ...} under "range_violations".
- Domain Validation
    - generic_domains specify allowed categories, e.g.:

        - Gender ∈ {Male, Female}
        - Churn ∈ {0, 1}
    - Unexpected values are listed under "domain_violations" per column with allowed and unexpected values.
- Outlier Detection (IQR)
    - iqr_stats(series) applies the IQR method to numeric columns (skips binary and too-short series) and reports count and bounds per column under "iqr_anomalies".
- Summary and Detailed Report Tables
    - summarize(report) gives a one-row summary with rows, cols, number of columns having missing values, duplicate counts, columns with range/domain issues, and columns with IQR outliers.
    - flatten_for_table(name, report, resolutions) expands all checks into a long-form table with dataset, check type, column, value, and details (including optional resolution notes).
    - Both summary and detailed tables are displayed; optional file writing hooks are present (WRITE_FILES and OUTDIR).

4.2 Data quality reporting tooling

- The script builds custom in-notebook reports (summary and detailed tables) and supports saving CSV reports via hooks.

5. Data Preparation

The code does the data preparation and transformation stage and is integrated around the Generic dataset.

5.1 Handling missing values (imputation/removal)

- Numeric casting via to_numeric_inplace ensures invalid entries are coerced to NaN; downstream logic is designed to avoid failures (e.g., safe divisions).
- For features derived from numeric columns, protective strategies are applied:
    - _safe_div and guards prevent divide-by-zero and replace non-finite results with 0 or NaN, depending on the step.

5.2 Standardization of numerical attributes

- Two scalers are used:
    - MinMax scaling: add_minmax creates -scaled versions with suffix _minmax for feature list candidates (e.g., months_active, spend_per_month, usage_per_month, support_calls_per_month, payment_punctuality, services_count).
    - Standardization: add_scaled applies StandardScaler to continuous variables and outputs z-score standardized columns with suffix _z.
- This ensures comparable ranges and scale normalization for downstream modeling.

5.3 Encoding of categorical variables

- The code performs feature engineering preparations and scaling; categorical encoding is implied by the presence of "numeric flags" and service-like pattern detection.

6. Feature Engineering & Transformation

The code implements an extensive feature engineering workflow for the Generic dataset and persists transformed data to a relational store.

6.1 Aggregation of customer behavior features

- Aggregate features are created with aggregate_generic:
    - spend_sum: sum of spend_per_month per CustomerID
    - calls_mean: mean support_calls_per_month per CustomerID
- The aggregated table is merged back to the row-level feature table for modeling enrichment.

6.2 Generation of derived features (tenure, frequency, spending, recency)

- build_generic_features constructs multiple derived signals:
    - months_active: numeric tenure proxy from Tenure
    - tenure_bucket: categorical bucketing of tenure (new/mid/loyal)
    - spend_per_month: from MonthlyCharges, or Total Spend / Tenure when MonthlyCharges are absent
    - usage_per_month: Usage Frequency / Tenure
    - support_calls_per_month: Support Calls / Tenure
    - payment_punctuality: 1 - normalized Payment Delay (higher is better)
    - services_count: sum across detected service-like numeric flags
    - last_interaction_year, last_interaction_month, days_since_last_interaction (recency features)
- The code safeguards against zero divisions and non-numeric values and uses coercion/guards to keep features consistent.
- Additional scaling:
    - MinMax-scaled variants for key engineered features
    - Standardized variants for a broader set of continuous variables

6.3 Storage in a feature store (retrieval-ready)

- Feature persistence to a relational database (SQLite) is implemented:
    - generic_features table written via SQLAlchemy (engine = create_engine(DB_URL))
    - DDL exported from sqlite_master for documentation (schema_*.sql)
    - Example analytical SQL queries are generated and saved (sample_queries_*.sql), and executed to validate content.
- This serves as a lightweight "feature store":

- Centralized, queryable feature table (generic_features)
- Versioned outputs (timestamped CSVs and DB artifacts)
- Reusable across training and evaluation steps

7. Feature Store

This section shows the implementation and usage of the **custom feature**. The feature store provides structured, versioned storage and retrieval of engineered features to support reproducible machine learning workflows.

7.1 Design and Structure

- **Storage Location:**
  The feature store is implemented as a directory-based repository in Google Drive at:
  /content/drive/MyDrive/DMML_Project/data/feature_store
- **Directory Layout:**

  feature_store/

  ├── feature_data/ # Contains feature snapshot CSV files

  │ ├── generic_features_YYYY-MM-DD_v1.csv

  │ └── ...

  ├── feature_metadata.csv # Metadata describing snapshots and features

  └── reports/ # Reports produced from retrieval operations

  └── retrieved_features_sample.csv

- **Naming Convention:**
  Feature snapshots are stored as CSV files named with a prefix, an as-of date, and a version suffix, e.g.:
  generic_features_2025-08-22_v1.csv
- **Entity Key:**
  The unique key for feature retrieval is configurable (default: row_id).

7.2 CustomFeatureStore Class (custom_feature_store.py)

- **Initialization Parameters:**
  - repo_dir: Root directory of the feature store repository.
  - entity_key: Column name used as the unique identifier for entities.
  - snapshot_prefix: Base name used for feature snapshot files.
- **Key Methods:**

    - Metadata (): Reads and returns the feature metadata as a DataFrame, enabling governance and auditing of available snapshots.
    - get_features(entity_ids=None, as_of=None, columns=None): Retrieves features filtered by entity IDs, snapshot date (as_of), and selected columns.
        - If the as_of snapshot file is missing, it falls back to the latest snapshot.
        - Filters for the specified entity IDs and columns. Returns a clean DataFrame suitable for modeling or analysis.
- **Internal Helpers:**
    - _latest_snapshot_path(): Returns path to the most recent snapshot file based on name sorting.
    - _snapshot_for_date(as_of): Constructs the file path for a given as-of date snapshot.

7.3 Retrieval Script (retrieve_features.py)

- Demonstrates the usage of CustomFeatureStore by:
    - Loading metadata and printing it for inspection.
    - Reading a snapshot CSV to obtain sample entity IDs.
    - Retrieving a subset of columns for selected entities as of a particular date.
    - Printing retrieved features to the console.
    - Saving retrieved features as a CSV report for documentation.

This task provides:

- **Storage in a Feature Store:**
    - The solution provides a versioned feature store persisted as CSV snapshots organized by date and version.
    - Enables time-travel and historical retrieval of feature sets to guarantee reproducibility.
- **Efficient Retrieval:**
    - Supports filtered queries by entity identifiers and selective feature columns.
    - Gracefully handles missing snapshots by defaulting to the latest available.
- **Governance and Metadata:**
    - Maintains metadata in a dedicated CSV to track feature snapshot versions and lineage.
- **Integration:**
    - Easily integrates into the ML pipeline, with engineered features saved during transformation and subsequently retrieved for model training or evaluation.

8. Feature Versioning and Tracking

This part documents the implementation of feature versioning and tracking in the project, ensuring reproducibility, auditability, and clear lineage of feature transformations.

The code:

- Maintains multiple versions of engineered feature snapshots to support rollback and debugging.
- Tracks changes and additions to features explicitly in metadata with version identifiers.

- Provides evidence of feature evolution with side-by-side comparisons and reporting.

8.1 Implementation Details

8.1.1 Snapshot Versioning

- Feature snapshots are stored as CSV files named with a consistent schema including a snapshot prefix, ingestion date, and version suffix, e.g.:
    - generic_features_2025-06-30_v1.csv
    - generic_features_2025-06-30_v2.csv
- The script automatically detects the latest v1 and v2 snapshots by filename if explicit dates are not found, ensuring flexibility.
- The v2 snapshot:
    - Introduces a new derived feature services_per_tenure (services_count divided by tenure, clipped properly to avoid division errors).
    - Modifies the existing feature services_count_minmax by increasing its value by 0.05 if PhoneService == 1, clipped between 0 and 1.

8.1.2 Metadata Management

- Feature metadata is stored in feature_metadata.csv and updated with each version:
    - Existing features with logic changes have their version bumped (e.g., services_count_minmax updated to version 2.0.0 with description).
    - New features are added with version 1.0.0, source description, and data type annotations.
- The metadata file supports keeping a comprehensive record of feature definitions, versions, and provenance.

8.2 Reporting and Evidence

- A report comparing the number of rows, added/removed columns between v1 and v2 snapshots is generated.
- Sample value changes for updated features are displayed side-by-side.
- The updated metadata table head is displayed.
- A textual version of this summary is saved to versioning_demo_output.txt.
- A markdown report, Feature_Versioning_Report.md, documents the rationale, naming conventions, and evidence files.

8.3 Packaging for Submission

- Key files packaged for submission include:
    - Feature snapshots (v1 and v2 CSVs)
    - Updated feature metadata CSV
    - Versioning report text file
    - Markdown report summarizing the feature versioning approach and artifacts
- A manifest file lists the included files, sizes, and MD5 hashes for integrity verification.

9. Experimentation with ML Algorithms

- Algorithms Used:
    - Logistic Regression with balanced class weights and solver liblinear, and a high max_iter to ensure convergence.
    - Random Forest Classifier with a large number of trees (n_estimators=30000), balanced subsampling for class weight, and parallel computation (n_jobs=-1).
- Training Pipeline:
    - Features and target label are loaded from the latest feature snapshot, including preprocessing (one-hot encoding for categoricals, filling NaNs, and infinities).
    - The dataset is split into training and test sets using stratified sampling to maintain class distribution.
    - Models are trained on the training set and evaluated on the test set.

9.1. Evaluation Metrics

- Metrics calculated to assess model performance include:
    - Accuracy
    - Precision
    - Recall
    - F1-score
    - ROC-AUC (if probability estimates are available)
- Additional outputs:
    - Detailed classification report (per-class precision, recall, F1, support)
    - Confusion matrix to visualize true/false positives/negatives
- Reporting is printed to the console and saved as a JSON report file with all evaluation details, facilitating audit and review.

9.2. Model Tracking & Persistence

- Model Persistence:
    - Trained models are saved as pickle (.pkl) files in a dedicated models directory.
    - The filenames include the model type and timestamp for versioning.
- Metadata Tracking:
    - Alongside model files, a JSON report is saved, capturing:
        - Timestamp of training
        - Model parameters
        - Performance metrics and reports
        - Paths and checksums of feature snapshots used to train the model
        - Dataset sizes (training and testing)
        - List of feature columns used

10. Orchestration

Our pipeline is orchestrated with **Prefect 2** and executed in **Google Colab**, producing reproducible, versioned runs stored on **Google Drive**. The DAG enforces a clear, linear dependency chain that moves data from raw ingestion to deploy-ready model artifacts with observability at every hop.

10.1 DAG structure.

1. **Ingestion:** Fetch raw datasets from sources and store them under data/raw/ with versioned filenames.
2. **Validation:** Run quality checks (nulls, ranges, dtypes) and produce a validation report. The pipeline fails fast if validation errors exceed a threshold.
3. **Preparation**: Clean and standardize the data, saving results in data/validated/.
4. **Transformation:** Generate engineered features and save them into data/transformed/churn_features.csv.
5. **Feature Store:** Publish transformed features to a feature store snapshot (feature_store/feature_data/..._vN.csv) and update feature_metadata.csv.
6. **Versioning:** Version artifacts (data, models, reports) using Git/DVC. The flow can optionally push changes to a remote repository if credentials are configured.
7. **Model Training & Evaluation:** Train ML models (Logistic Regression, Random Forest), save them under models/, and produce evaluation JSON reports under reports/. The flow enforces a quality gate (ROC AUC ≥ threshold).
8. **Run Summary:** Collects all outputs (raw data, validated data, transformed data, feature snapshots, models, evaluation reports) into a single JSON summary under orchestration_artifacts/orchestration_run_summary.json.

10.2 Tool Used

We used Prefect for orchestration because it is lightweight, works well in Colab/local environments, and provides built-in support for task retries, logging, and scheduling. The flow executes notebooks (Points 1–6) and scripts (Points 7–9) in the correct order, with proper dependencies.

Benefits:

- **Automation:** No need to run each step manually; the entire workflow is triggered in one command.
- **Reproducibility:** Each run is versioned and produces a run summary artifact.
- **Reliability:** Validation and model quality gates ensure the pipeline fails safely when outputs are not trustworthy.
- **Extensibility:** Can be scheduled for periodic runs (daily/weekly) using Prefect Deployments or ported to Airflow/Dagster for production.

Challenges Faced and Solutions Implemented

Challenge	Description	Solution Implemented
Data Collection & Integration	Multiple data sources (Kaggle, Hugging Face) with varying formats, structures, and completeness.	Automated data ingestion scripts for each source using respective APIs (Kaggle, HF Datasets); consistent CSV conversion, scheduled ingestion, and explicit error logging for robustness.
Data Granularity & Consistency	Variation in timestamp granularity and feature formats across datasets.	Applied aggregation and transformation scripts (Task 6) to harmonize features and derived attributes, with schema validation and standardized raw storage in a partitioned, versioned structure.

Data Quality & Missing Values	The presence of missing values, duplicates, and inconsistent types hampered analysis and modeling.	Used pandas for missing value detection, type coercion, and safe imputation; validated datasets with custom data validation reports and provided hooks for Great Expectations integration.
Handling Categorical & High Cardinality	Categorical variables (e.g., customer segments) risked value explosion with naive encoding.	Feature engineering routines applied one-hot encoding (with pandas/sklearn) and aggregation methods to reduce dimensionality and create informative features.
Feature Drift & Data Drift	Potential for customer behavior change impacting model accuracy over time.	Measurement of drift and pipeline versioning through metadata tracking and engineered monitoring scripts; provision for model retraining as new data snapshots arrive.
Error Handling in Data Ingestion	API failures and partial/incomplete ingestions disrupted reliable data flow.	All ingestion functions use try/except blocks with error logging and batch status reporting in log files; robust fallback logic selects valid snapshots if the target is missing.
Pipeline Failures & Dependency Management	Complex dependencies between tasks (validation, transformation, feature store updates).	Modular code and clear folder partitioning ensure atomicity of each pipeline task; recommended use of orchestration tools (e.g., Airflow DAGs) for scalable production management.
Versioning & Reproducibility Issues	Feature definitions and datasets evolved, risking non-reproducible results.	Used timestamped snapshots and updated feature metadata with explicit versioning in CSVs; reports and manifests record lineage and checksum for every version; DVC usage recommended.

Model Overfitting & Generalization	Models performed well on training data but overfitted due to class imbalance or excessive features.	Incorporated regularization (logistic regression), dropouts (if using NN models), and cross-validation; maintained a balanced test/train split and reported comprehensive metrics per run.

Final Note & Execution

This pipeline demonstrates a complete journey from raw data ingestion to model training and evaluation. By integrating validation, feature engineering, versioning, and orchestration with Prefect, we built a workflow that is automated, reproducible, and reliable. The result is a scalable foundation for churn prediction that can be extended to future datasets and models with ease.

Contributors (100% Equal Contribution):

- Amulya Gupta (2024AB05200)
- Navya Saxena (2024AA05234)
- Ravi Bhardwaj (2024AA05356)

All members contributed equally to the design, coding, documentation, and evaluation of this pipeline.

Drive Folder:
https://drive.google.com/drive/folders/1SHL2iUlX6lTv1SNqXRNekuG73oOsDj_i?usp=sharing

Screenshot:
https://docs.google.com/document/d/1fHy64msPSO31HLDYLs7rf4QYJAurp-0H8uXY5dmYqYs/edit?usp=sharing